

# AI-Powered SaaS Platform for Document-Based Chatbot Agents using Retrieval-Augmented Generation (RAG)

## Project Proposal

Duration: 6 Months

Prepared on: 2025-07-31

### Abstract

This project aims to build a scalable web-based SaaS platform that enables businesses to create their own AI chatbot agents from uploaded PDF knowledge bases using Retrieval-Augmented Generation (RAG). The system integrates authentication, subscription plans with free trials, document ingestion, vectorization using Pinecone, and a conversational interface powered by OpenAI. Users can integrate the resulting chatbot agents into their own applications (e.g., Mobile apps & Web apps) via secure API keys.

### Objectives

- Design and implement a secure user onboarding and subscription system with a 7-day free trial.
- Allow businesses to upload and manage PDF documents as knowledge sources.
- Process and chunk document content, create embeddings, and store them in Pinecone for efficient retrieval.
- Provide a RAG-based chatbot agent interface that answers queries using document context.
- Offer integration mechanisms (API, Mobile apps & Web apps, code snippets) for external applications.
- Support document updates with versioning and automatic regeneration of chatbot knowledge.
- Ensure authentication, API key management, and secure access for integrations.

### Problem Statement

Many businesses have internal or external document repositories (PDFs) that are underutilized because extracting actionable insights requires manual search or static FAQs. This solution provides a dynamic conversational interface that understands the content of uploaded documents and serves as a customizable knowledge assistant across platforms.

### Proposed Solution

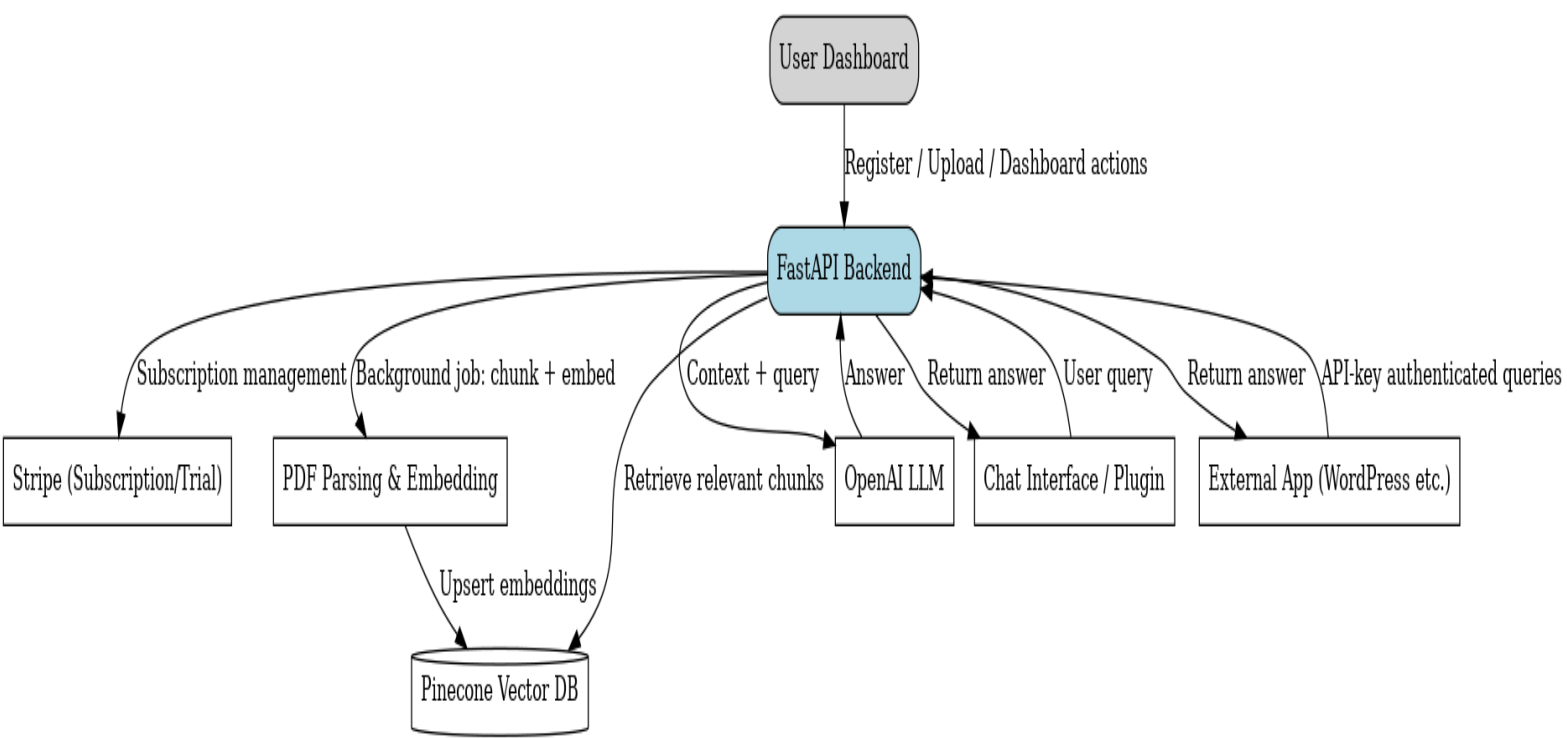
Build a SaaS platform where users can upload PDF documents and immediately get an AI chatbot agent that understands those documents. The backend processes PDFs, converts them into embeddings, stores vectors in Pinecone, and serves a conversational interface using OpenAI. The platform includes subscription handling, API key-based access, external integration options (like Mobile apps & Web apps), and document versioning.

# Technology Stack & Required Skills

| Layer                     | Technology / Tools                  | Skills Needed                                             |
|---------------------------|-------------------------------------|-----------------------------------------------------------|
| Frontend                  | React, TailwindCSS, Next.js         | JavaScript/TypeScript, UI/UX, embedding chat widge        |
| Backend                   | Python, FastAPI                     | Async programming, REST, auth, background                 |
| Vector Database           | Pinecone                            | Vector similarity, metadata filtering                     |
| Embeddings & LLM          | OpenAI API                          | tasks Prompt engineering, handling context, rate limiting |
| Authentication & API Keys | JWT, secure key storage             | Security, hashing, key rotation                           |
| Payments                  | Stripe (subscription + 7-day trial) | Webhook handling, subscription logic                      |
| PDF Processing            | PyMuPDF / pdfplumber / LangChain    | loadersText extraction, chunking                          |
| Storage                   | AWS S3 or Firebase                  | Cloud storage integration                                 |
| Deployment                | Vercel / Railway / AWS              | CI/CD, environment management                             |
| Integration               | Mobile apps & Web apps              | API consumption                                           |

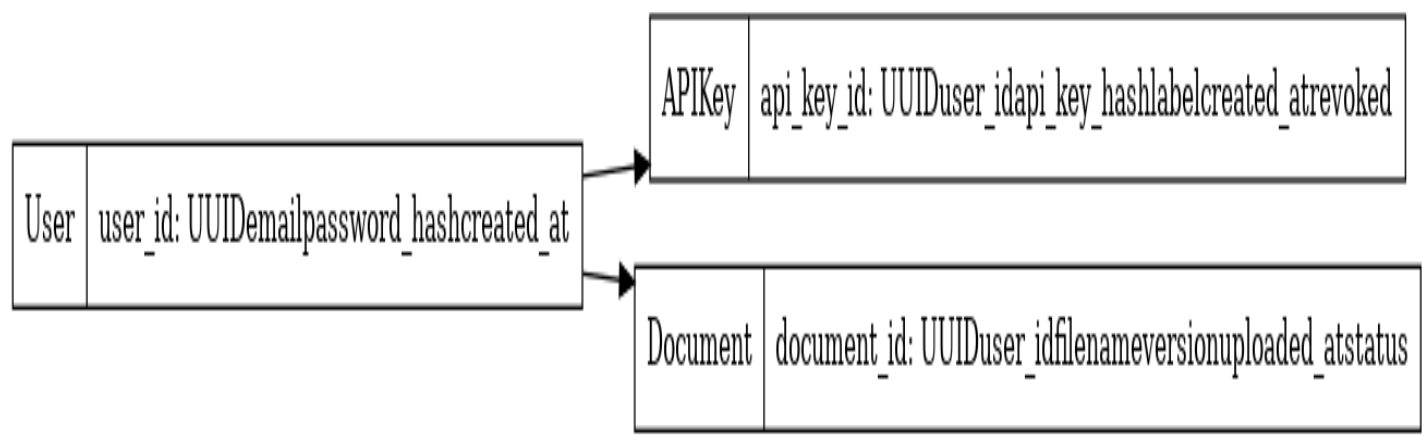
# System Architecture and Data Flow

The following diagram illustrates the components and the flow of data from user registration through PDF ingestion to chatbot interaction and external integration.



# Entity-Relationship Diagram (ERD)

Core data models and their relationships.



## Implementation Plan & Milestones (6 Months)

- Month 1: Finalize requirements, scaffold project (FastAPI backend, frontend), set up authentication and Stripe integration.
- Month 2: Implement PDF upload, processing, embedding pipeline, and Pinecone integration; begin basic chat interface.
- Month 3: Develop full RAG pipeline, improve context assembly, and integrate OpenAI; build dashboard and API key management.
- Month 4: Create external integration components (Mobile apps & Web apps, SDK snippets), refine UI/UX, and add document versioning.
- Month 5: Stress test, add rate limits, implement monitoring, handle edge cases, secure the platform.
- Month 6: Final testing, prepare documentation, deploy production version, and collect user feedback.

## Expected Outcomes

- A fully functional SaaS RAG chatbot agent platform with subscription and integration capabilities.
- Secure API key-based access for external applications.
- Document management with versioning and dynamic chatbot updates.
- Reusable integration SDKs and Mobile apps & Web apps.
- Scalable architecture capable of supporting multiple businesses simultaneously.

## Future Scope

- Support for more document types beyond PDF (e.g., Word, HTML).
- Multi-language support for uploaded content and chatbot agent responses.
- Analytics dashboard showing user query trends, document usage, and satisfaction.
- Self-hosted options or white-labeling for enterprise clients.
- Fine-tuning domain-specific models for better relevance.

## Hardware / Software Requirements

- Cloud hosting account (AWS, Firebase, or similar) for backend and storage.
- Pinecone account for vector database.
- OpenAI API access with appropriate rate limits/plan.
- Stripe account for subscription management.
- Domain and SSL for secure access.
- Development machines with Python, Node/JS toolchain for frontend.

## Risk Analysis & Mitigation

Cost Overruns: Embedding and vector search costs may grow; mitigate with quotas/usage alerts.

Data Leakage: Ensure encryption, isolate user data, and allow revocation of keys.

Model Hallucination: Include source attribution and fallback responses.

Integration Breakage: Provide clear SDKs and versioned APIs for external consumers.